

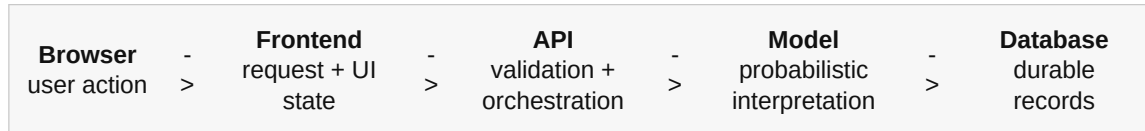
Production Reliability & Observability

Browser-to-database failure boundaries, logs and traces, safe retries, latency, configuration, and graceful degradation.

Core idea

A production AI failure can start anywhere in the chain. Debug from the request forward and find the **first divergence**, not just the final bad answer.

The production path



What can fail at each boundary

Boundary	Typical failure	First evidence to inspect
Browser / frontend	Stale local state, rerender race, bad API base, double submit	Network request, console, rendered state, reproduction video.
API / app logic	Wrong status, unsafe retry, bad validation, duplicate side effect	Request ID, route logs, response body, persisted records.
Model provider	Timeout, refusal, malformed/poor interpretation, latency spike	Provider timing, model ID, token counts, safe error category, structured result.
Database	Constraint failure, stale version, transaction rollback, migration drift	Transaction logs, row versions, migration state, affected records.
Cross-boundary	Works locally, fails deployed; config or environment mismatch	Environment variables, deployed commit, service logs, DB target, CORS.

Observability: trace, log, metric

Signal	Answers
Trace / request journey	What happened for this one request, in what order?
Log	What technical event, warning, or error happened?
Metric	How often, how slow, how expensive, or how widespread is it?
Correlation / request ID	Which browser request, API logs, provider call, and DB activity belong together?

Production Reliability & Observability

Browser-to-database failure boundaries, logs and traces, safe retries, latency, configuration, and graceful degradation.

Safe failure and retry design

AI calls are slow and fallible. The application should assume they can fail after partial work has already happened.

Situation	Bad recovery	Better recovery
Evidence saved; model call fails	Tell user to submit the Note again	Keep the same Evidence and offer retry analysis.
Review accept races with newer State	Overwrite newer truth	Reject stale proposal using version checks.
Model returns valid shape but bad references	Persist partial downstream records	Reject deterministically; preserve the failed interpretation record.
Frontend waits for server update	Sleep 300 ms and hope	Use the authoritative response, then deterministic revalidation.
Provider unavailable	Expose internal exception or mutate State anyway	Return a safe service failure; preserve source Evidence; no consequential mutation.

Latency: measure the parts, not just the total

Total latency = frontend/network + API work + provider call + validation/persistence + rehydration.
If the model takes 15 seconds and persistence takes 20 ms, optimizing SQL will not fix the user experience.

Production checklist

- **Identity:** know the deployed version/commit and model identifier.
- **Configuration:** confirm API base, provider keys, CORS, database URL, and environment-specific settings.
- **Durability:** know which records must survive restart/redeploy and where they are stored.
- **Retries:** decide which operations are safe to retry and which require idempotency or version checks.
- **Logging:** record safe request IDs, timing, model/provider, token usage, and categorized errors - not hidden reasoning.
- **Regression:** every production defect should become a test/eval case when feasible.
- **Graceful degradation:** if AI is unavailable, preserve authoritative data and explain what did not complete.

One-sentence summary

Reliable AI products make every boundary observable, every consequential write controlled, and every retry safe.